Apache Airflow

Cheat Sheet

Setup • Concepts • DAGs • Connections • Docker • S3 • Hooks

01 Run Airflow in a Python environment

```
python -m venv <name-env>
```

Create a virtual environment

```
<name-env>\Scripts\activate
```

Activate the virtual environment (Windows)

```
$env:AIRFLOW_HOME = "."
```

Change the directory where the database is created

```
airflow db init
```

Initialize the database (SQL, logs folder and some config files)

```
airflow users create --username <username> --firstname <firstname> --lastname <lastname>
--role Admin --email <email>
```

Create an admin user (password used here: airflow)

```
airflow webserver -p 8080
```

Start the web server

```
airflow scheduler
```

Start the scheduler (the AIRFLOW_HOME variable may need to be set again)

02 Run Airflow in Docker

```
curl -LfO "https://airflow.apache.org/docs/apache-airflow/2.10.0/docker-compose.yaml"
```

Download the docker-compose YAML file

Note Replace CeleryExecutor with LocalExecutor, and comment out all Celery-related lines.

```
mkdir logs plugins dags
```

Create the required directories

```
docker compose up airflow-init
```

Initialize the database

```
echo AIRFLOW_UID=50000 > .env
```

Set the user id (removes a warning, not strictly necessary)

```
docker compose up
```

Run the containers

03 Airflow concepts

- **DAG** (Directed Acyclic Graph): a workflow, i.e. a set of tasks (nodes) with dependencies between them.
- **Upstream / downstream**: in A → B → C, B is upstream of C and downstream of A.
- **Operators** determine what a task does: BashOperator, PythonOperator, custom operators...
- **Execution date**: the logical date and time of a run.
- **Task instance**: a run of a task at a specific execution date.
- **DAG run**: an instantiation of a DAG containing task instances.

Task lifecycle

```
no_status        empty task instance
  |-> scheduled
        |-> executor puts the task in the queue  ->  queued
              |-> queued   (executed when a worker is free)
                    |-> worker
                          |-> running -> (sometimes) up_for_reschedule
                          |      (task re-checked every time interval)
                          |-> success
                          |-> failed  -> up_for_retry
                          |-> shutdown

Other states:  removed | upstream_failed | skipped
```

04 Basic architecture

- The **webserver**, **scheduler**, **executor** and **worker** all have access to the database and the DAGs.
- The data engineer configures `airflow.cfg`, can access everything through the user interface, and is the author of the DAGs.

05 Airflow DAGs

```
docker compose down -v
```

Shut down the containers as well as the volumes

```
AIRFLOW__CORE__LOAD_EXAMPLES: "false"
```

Set to false in the YAML file

```
docker compose up airflow-init
```

```
docker compose up -d
```

- Create your first DAG (a Python file) in the dags folder, with a BashOperator.
- Do the same with a PythonOperator.
- Create tasks with the TaskFlow API (see the Python file).

06 Backfill and catchup

Catchup: see the Python file.

Backfill

```
docker exec -it <id_container> bash
```

Enter the bash of the scheduler container

```
airflow dags backfill -s <start-date> -e <end-date> <dag_id>
```

```
exit
```

Leave the container bash

07 Scheduler with cron expressions

Use **crontab.guru** to build and check expressions: https://crontab.guru/

08 Database connection

Airflow can connect to database servers, cloud servers and more. Credentials (host, user, password...) are managed as Airflow **Connections** through the UI.

Setup with Postgres in Docker

- In the docker-compose YAML file, expose the port of the postgres container: `5432:5432`.
- Connect to it with **DBeaver** (a cross-platform database management tool): create a Postgres connection, then a new database called `test`.
- Create the DAG file `dags_with_postgres_operator`.
- Create the connection through the Airflow web UI.

```
docker compose up -d --no-deps --build postgres
```

--no-deps: do not start the services it depends on

Remark Since Airflow runs in Docker, put host.docker.internal in the Host field of the connection. See the Python file for more details.

09 Install Python packages in the Airflow Docker image

There are two ways to add Python dependencies to the Airflow container:

- **Image extending**: basic Docker knowledge is enough.
- **Image customising**: needed to optimize the image size, for example.

Method 1: Image extending

- Create `requirements.txt` containing the dependency, e.g. `scikit-learn==0.24.2`.
- Create a `Dockerfile` that extends the official Airflow image: copy the txt file into the image, upgrade pip, then run `pip install` on the requirements.

```
docker build . --tag extending_airflow:latest
```

Build the extended image

- In the YAML file, change the image tag to `extending_airflow:latest`.
- Create a DAG file to verify that scikit-learn is installed.
- Restart the webserver and scheduler, since the image name changed:

```
docker compose up -d --no-deps --build airflow-webserver airflow-scheduler
```

Note To add a new dependency to requirements.txt later, rebuild and run the same command again.

Remark In my case I had to create a .wslconfig file to allocate more memory to Airflow (at least 4 GB). Then restart with: wsl --shutdown

```
[wsl2]
memory=4GB       # at least 4GB for Airflow
processors=2     # adjust the number of CPUs if needed
```

Remark I also tried changing the webserver port to 8081 (because of an error message). It did not work in my case, so I used the 2nd method.

Method 2: Image customising

The image must be built from the Airflow source code.

```
cd desktop
```

```
git clone https://github.com/apache/airflow.git
```

Clone the Airflow source code

- Open the airflow folder and locate `docker-context-files`. Python dependencies are installed from there.
- Create the `requirements.txt` file inside it.

```
docker build . --build-arg AIRFLOW_VERSION='2.0.1' --tag customising-airflow:latest
```

- Replace the image name in the YAML file with `customising-airflow:latest`, then restart the services:

```
docker compose up -d --no-deps --build airflow-webserver airflow-scheduler
```

10 AWS S3 sensor operator (with MinIO)

A **sensor** is a special kind of operator that waits for something to happen, working like a trigger. Here we use **MinIO**, an open-source, S3-compatible object store, run as a Docker service.

Run MinIO

```
docker run
  -p 9000:9000
  -p 9001:9001
  --name minio1
  -e "MINIO_ROOT_USER=ROOTUSER"
  -e "MINIO_ROOT_PASSWORD=CHANGEME123"
  quay.io/minio/minio server /data --console-address ":9001"
```

- Open the console (web UI) via localhost and log in.
- Create a bucket called `airflow`.
- Create a `data` folder with a `data.csv` file, then drag it into the bucket.
- Build a DAG that connects to the bucket and senses the file's existence.

Connect Airflow to MinIO

MinIO is S3 API compatible, so the AWS S3 API from the Amazon Airflow provider package can be used. Open the scheduler container bash and check the version:

```
pip list | grep amazon
```

In our case: 8.27.0

Remark The S3 connection type was not showing. Fix: install the provider, then restart the services.

```
pip install apache-airflow-providers-amazon
```

```
docker compose up -d --no-deps --build airflow-webserver airflow-scheduler
```

If S3 is still not in the list, choose HTTP or Amazon Web Services and configure the connection manually. In the **Extra** field, add:

```
{"aws_access_key_id": "ROOTUSER",
 "aws_secret_access_key": "CHANGEME123",
 "host": "http://host.docker.internal:9000"}
```

Note poke is the default mode of sensor operators: it checks for the file's existence every poke interval, within the configured timeout limit.

11 Hooks: S3 and PostgreSQL

Goal: import an `orders.csv` file into a Postgres database, then write a DAG that queries the data and uploads it to the S3 bucket.

Steps

- Connect to DBeaver, create the `orders` table and import `orders.csv`.
- Write the DAG file (see the Python file).

A **hook** is a component that lets us interact with external systems without worrying about the implementation. To query data we need the **Postgres hook**.

Finding the right hook API

- Check the version of the installed postgres package from the scheduler container bash:

```
pip list | grep postgre
```

- On the Airflow documentation site, search the Postgres providers section, click *Python API*, then the *hooks* subpackage.
- A connection can be established via the `get_conn` function; credentials must be provided to connect the hook.
- To upload to the S3 bucket, check that the S3 hook is compatible with the installed Amazon provider version.